A Comparative Study of SGD, Adam, AdamW, and Muon Optimizers: Convergence, Generalization, and Computational Efficiency

Anonymous public version Student ID: Omitted for privacy
CIE6007 Machine Learning, 2026–27 Term 1
Code and results: `https://github.com/XFDG/cie6007-optimizer-benchmark`

Abstract

We compare momentum SGD, Adam, AdamW, and a Muon–AdamW hybrid on Fashion-MNIST using a fixed 335k-parameter MLP. Three learning rates per method are tuned on held-out training data; selected settings are evaluated with three paired seeds over 15 epochs. Muon+AdamW attains the highest mean test accuracy, 89.04 ± 0.29%, using validation-selected checkpoints. Muon uses 1.97× AdamW's training time and differs in test accuracy by +0.45 percentage points. AdamW reaches 85% validation accuracy fastest on average. Equal-step accuracy and time efficiency are reported separately, with a predeclared five-second trajectory readout. These measurements characterize one small MLP and one implementation; they do not establish a universal optimizer ranking.

1 Introduction

The optimizer controls how a neural network converts gradients into parameter updates. Momentum SGD accumulates a search direction, while Adam rescales coordinates using gradient moments [1, 2]. AdamW separates parameter shrinkage from the gradient transformation [3]. Muon instead approximately orthogonalizes matrix updates [4]. This offers a different way of allocating updates across directions, but introduces additional matrix multiplications and requires an auxiliary optimizer for other parameters.

An optimizer can improve accuracy per training step without improving accuracy per second. A recent paired-budget study illustrates that these rankings can differ across architectures [5]; its transformer results do not establish a ranking for a small multilayer perceptron (MLP). We therefore ask four questions: which method converges fastest at equal steps (RQ1), generalizes best (RQ2), offers the strongest measured time–accuracy trade-off (RQ3), and is most sensitive to the tested learning rates (RQ4)? We compare actual training runs without assuming Muon must win. The contribution is a reproducible, budget-matched case study, rather than a new optimization algorithm.

2 Methods and Algorithms

Let $g_t = \nabla \ell_t(\theta_t)$ and let λ apply only to weight matrices, with zero decay for biases. Our momentum SGD baseline uses

$$u_t = g_t + \lambda\theta_t, \qquad b_t = 0.9b_{t-1} + u_t,$$
$$\theta_{t+1} = \theta_t - \eta b_t. \tag{1}$$

Adam forms exponential moving averages m_t and v_t of u_t and u_t^2, with $(\beta_1, \beta_2) = (0.9, 0.999)$ and bias correction denoted by hats:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + 10^{-8}}. \tag{2}$$

AdamW computes these moments from g_t and applies

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + 10^{-8}}. \tag{3}$$

Thus Adam's coupled L_2 penalty changes its adaptive moments; AdamW's decay does not. The same numerical $\lambda = 10^{-4}$ is a controlled configuration choice, not matched effective regularization.

For Muon, we use unmodified `torch.optim.Muon` from PyTorch 2.11 [6]. It uses momentum 0.95, Nesterov extrapolation, and five Newton–Schulz iterations. Given the normalized momentum-derived matrix X, each iteration computes

$$A = XX^\top, \qquad X \leftarrow aX + (bA + cA^2)X, \tag{4}$$

with $(a, b, c) = (3.4445, -4.775, 2.0315)$. These iterations approximately reshape the singular values; five iterations do not guarantee exact orthogonality. The native implementation uses BF16 internally for this operation; model parameters and forward/backward computation remain FP32. We use `adjust_lr_fn='original'`, which gives unit shape scaling for our square hidden matrices.

Parameter routing. The MLP has dimensions $784 \to 256 \to 256 \to 256 \to 10$, ReLU after each hidden layer, and 335,114 parameters. Muon updates only the two 256×256 hidden-to-hidden matrices (131,072 parameters, 39.1%). Input projection, classifier, and every bias use auxiliary AdamW with fixed learning rate 10^{-3} and the same decay policy. The comparison is therefore between practical optimizer configurations: the Muon treatment is a hybrid. Routing and auxiliary updates are included in all reported timing. SGD, Adam, and AdamW each update the whole model.

3 Experimental Setup

Data and pairing. Fashion-MNIST [7] has 60,000 official training and 10,000 official test examples. We hold

Table 1: Accuracy at validation-best checkpoints, with final-epoch test accuracy for comparison. Mean ± sample SD across three paired seeds; all accuracies and gaps are in percentage points. Muon+ denotes the hybrid.

Method	Selected LR	Train	Validation	Test	Train–test gap	Final test
SGD-M	0.03	91.08 ± 0.77	88.82 ± 0.15	87.57 ± 0.32	3.51 ± 0.52	87.52 ± 0.51
Adam	0.001	92.01 ± 0.62	88.97 ± 0.47	88.32 ± 0.40	3.70 ± 0.25	88.05 ± 0.41
AdamW	0.001	92.68 ± 0.52	89.33 ± 0.15	88.59 ± 0.18	4.09 ± 0.35	88.55 ± 0.30
Muon+AdamW	0.005	94.03 ± 1.21	89.73 ± 0.21	89.04 ± 0.29	4.99 ± 0.92	88.97 ± 0.14

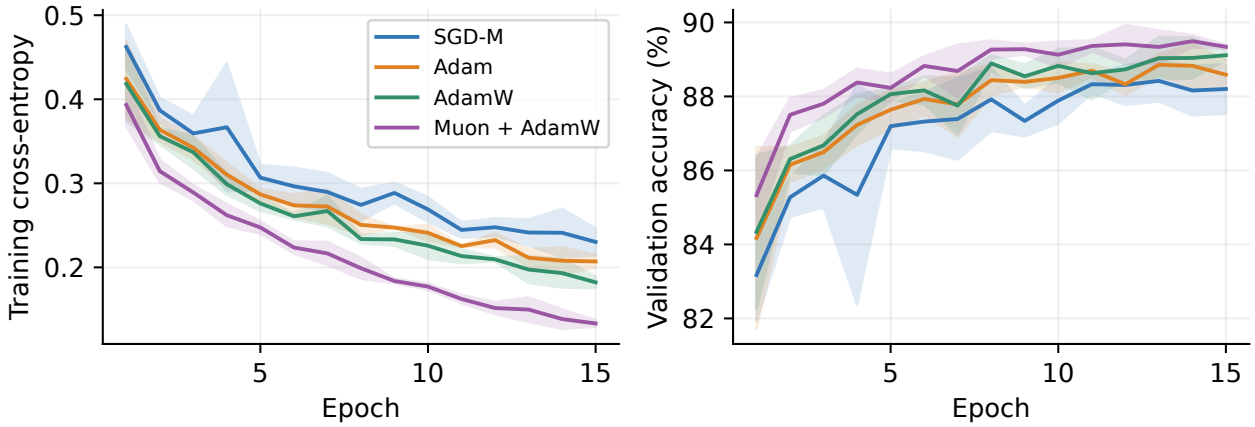


Figure 1: Training-set loss at epoch-end checkpoints and validation accuracy at equal epochs/updates. Lines are means; bands show one sample standard deviation over three seeds.

out 1,000 training examples per class for validation (split seed 2026), leaving 50,000 for training. Pixels are divided by 255; no statistics are fitted, and there is no augmentation or dropout. Batch size is 128, including the last partial batch: 391 updates per epoch. All configurations for a seed share identical initial weights and batch permutations. The test tensors are loaded only by a separate final-evaluation script, after tuning and all final training are frozen.

Equal tuning budget. Each optimizer receives three learning rates and ten epochs per candidate, using development seed 17. SGD tests $\{.01, .03, .1\}$; Adam and AdamW test $\{.0003, .001, .003\}$; Muon tests $\{.005, .02, .05\}$. We select the highest validation accuracy attained, breaking ties by lower validation loss then lower rate. Auxiliary Muon AdamW settings are fixed, so no extra search trials are granted. There is no learning-rate schedule. Selected configurations are independently rerun for 15 epochs (5,865 updates) with seeds 101, 202, and 303. Reported standard deviations use $n - 1$ in the denominator. Three seeds quantify run variability only weakly and are not evidence of a universal ranking.

Checkpoints and timing. Primary test accuracy uses the validation-best checkpoint; final-epoch accuracy is also reported to preserve the equal-step comparison. A common 85% validation threshold was specified before tuning. Threshold time is recorded at the first epoch-end observation, without interpolation; a missing crossing is censored, not zero.

Experiments execute sequentially on one NVIDIA H200 (driver 580.105.08), Python 3.12.3, PyTorch 2.11.0+cu130, and CUDA 13.0. Twenty disposable training steps warm each configuration; model, optimizer, and RNG are then recreated. We synchronize CUDA around training and validation. Datasets are GPU-resident. Training time includes minibatch indexing, forward, backward, and all optimizer updates; data loading, warmup, extra train-set evaluation, and checkpoint I/O are excluded. No compilation, autotuning, AMP, or TF32 is used. GPU peak memory is process tensor allocation, including resident data, rather than total device use. Other GPUs retain keep-alive workloads; GPU 0 has no keep-alive competitor. Shared CPU resources and hardware scheduling remain possible noise sources.

Common-time readout. After the smoke test and before full-data training, a five-second cumulative *training plus validation* budget was fixed. We select the validation-best completed epoch available by that budget from each trajectory. This is an epoch-boundary retrospective readout, not a separately interrupted end-to-end timed run. Unused time within the next epoch and excluded reporting overhead limit its interpretation as a strict wall-clock comparison.

4 Results and Discussion

Accuracy and generalization (RQ2). Table 1 reports validation-selected checkpoints. Muon+AdamW has the highest mean test accuracy ($89.04 \pm 0.29\%$). All values use three seeds, not repeated evaluations of one model. The final-epoch column checks whether check-

Table 2: Measured efficiency. Train seconds exclude validation; threshold seconds include it. Throughput is in thousands of training samples/s. Peak allocated MiB includes GPU-resident development data.

Method	Train (s)	Epoch (s)	k samples/s	Time to 85% (s)	Peak (MiB)
SGD-M	9.40 ± 0.21	0.63 ± 0.01	79.83 ± 1.80	1.43 ± 0.35	257.0 ± 0.0
Adam	11.31 ± 0.19	0.75 ± 0.01	66.31 ± 1.12	0.99 ± 0.42	258.3 ± 0.0
AdamW	11.16 ± 0.08	0.74 ± 0.01	67.20 ± 0.51	0.98 ± 0.43	258.3 ± 0.0
Muon+AdamW	22.04 ± 0.08	1.47 ± 0.01	34.03 ± 0.13	2.02 ± 1.00	257.8 ± 0.0

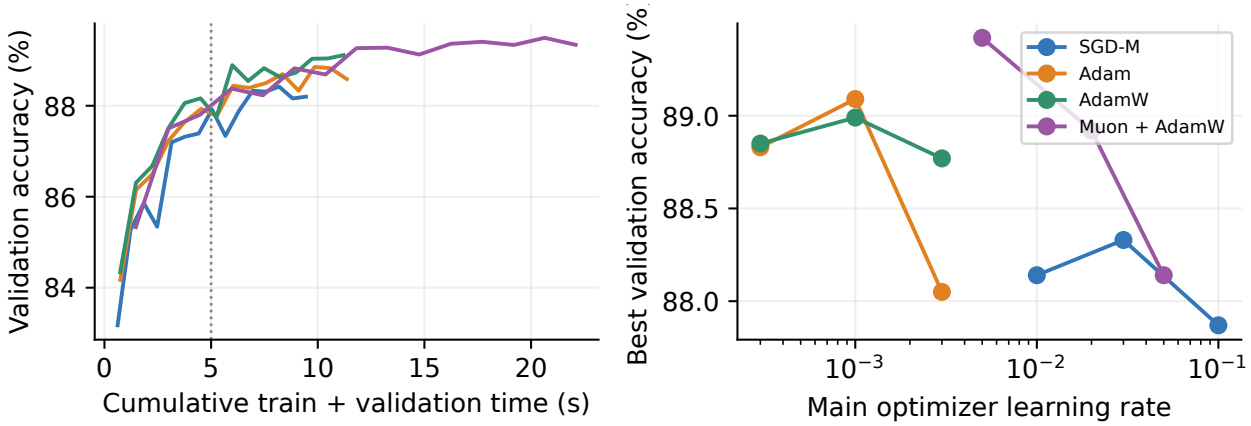


Figure 2: Left: validation accuracy against measured cumulative training-plus-validation time; dotted line is the five-second budget. Right: validation-best accuracy for each of three learning-rate candidates (one tuning seed, ten epochs). Muon changes only its matrix learning rate; auxiliary AdamW stays at 10^{-3}.

point selection changes the interpretation.

Convergence (RQ1). The first observed 85% validation crossing occurs at mean epochs 2.33 (SGD-M), 1.33 (Adam), 1.33 (AdamW), 1.33 (Muon+AdamW). AdamW is fastest by the corresponding training-plus-validation time. Figure 1 shows that ranking by early progress need not match ranking by the final checkpoint. Epoch-end training loss is evaluated on the full training subset, avoiding the changing-model average implicit in an online minibatch loss.

Computational efficiency (RQ3). SGD-M has the lowest 15-epoch training cost. Muon takes 1.97× AdamW time (22.04 versus 11.16 seconds). The extra matrix operations and auxiliary optimizer calls are plausible causes of the overhead, but epoch timing alone does not attribute cost to individual kernels. All methods have the same samples per epoch; throughput therefore expresses the same cost on a per-sample scale.

At the five-second readout, validation-best-so-far checkpoints achieve test accuracies of 86.94 ± 0.52% (SGD-M), 87.21 ± 0.56% (Adam), 87.54 ± 0.12% (AdamW), 87.18 ± 0.57% (Muon+AdamW). AdamW has the highest mean under this specific accounted-time budget. The budget selects different training durations; this is complementary to, rather than interchangeable with, the fixed-epoch comparison. A strict online deadline, data-loading cost, or a larger model could alter the ranking.

Learning-rate sensitivity (RQ4). Across the three tested rates, the best-validation range spans 0.46 points (SGD-M), 1.04 points (Adam), 0.22 points (AdamW), 1.28 points (Muon+AdamW). This range measures sensitivity within the specified grids, not over all rates. A single tuning seed can favor a configuration by chance; three equal trials do not guarantee equally comprehensive searches for different algorithms. Muon has an additional fixed auxiliary learning rate whose sensitivity is not explored. Its winning main rate, .005, lies at the lower grid boundary, so a better rate outside the grid remains possible.

Interpretation and limitations. Muon changes mean test accuracy relative to AdamW by +0.45 points while requiring 1.97× the training time. Its approximate orthogonalization changes update geometry, but that mechanism alone does not imply a gain on this dataset. Higher training accuracy is not sufficient evidence of better generalization: the train–test gaps in Table 1 must be read with test accuracy, not minimized in isolation. The gaps compare accuracies at the same checkpoint, not training accuracy averaged during optimization. The largest confounds are parameter routing (only 39.1% of parameters use Muon), fixed regularization strength, a short horizon, one dataset, and the small number of seeds. A hybrid-control experiment with identical auxiliary AdamW for every method would help isolate the hidden-matrix optimizer. The current experiment supports configuration-level conclusions only. Muon reaches 94.03% mean training accuracy with a 4.99-point train–test gap, compared with AdamW at 92.68% and 4.09 points. The additional fitting transfers only partly to test accuracy. Adam and AdamW cross 85% validation at nearly the same

measured time (0.99 versus 0.98 seconds), far smaller than their seed variation; this is not a meaningful speed separation. Sample SD reflects seed variation on one split and GPU; it excludes uncertainty over data splits, hardware, implementation choices, and hyperparameter search. No statistical significance or general superiority is claimed.

5 Conclusions

Under equal ten-epoch tuning trials and a 15-epoch final budget, Muon+AdamW obtains the highest mean validation-selected test accuracy, while SGD-M has the lowest training cost. Muon achieves 89.04 ± 0.29% test accuracy with an explicitly mixed parameter update scheme. Accuracy per step, accuracy per second, and learning-rate robustness provide different evidence; no single metric justifies choosing an optimizer for every task. Broader datasets, routing controls, and more tuning seeds would be the most useful next checks.

Code availability. Code, configurations, raw measurements, and reproduction instructions are available in the GitHub repository linked on the first page. The measured training implementation is archived at commit `3a281c2`; individual run records include source hashes.

AI usage and reproducibility. OpenAI Codex [8] substantially assisted protocol design, code generation, experiment execution, validation, plots, and drafting. Context7 and web/repository retrieval supported documentation checks. All numerical results come from the saved runs, not generated examples or borrowed measurements. Automated checks do not establish that the student has independently reviewed every claim. The repository includes configuration, source hashes, environment details, raw epoch CSVs, checkpoint evaluation hashes, and the development report. The student remains responsible for understanding and verifying the submission.

References

[1] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. arXiv:1412.6980v9, revised 2017.

[3] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. arXiv:1711.05101v3.

[4] Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. Technical blog and implementation, 2024. Code: github.com/KellerJordan/Muon; accessed 11 Nov. 2026.

[5] Mushir Akhtar, Hazem N. Nounou, Majdi Mansouri, and Mohamed N. Nounou. When does muon actually win? a paired data-, time-, and tuning-budget benchmark, 2026. arXiv:2610.21555v1.

[6] PyTorch Contributors. PyTorch 2.11: Muon optimizer documentation and source, 2026. Version 2.11.0+cu130; accessed 11 Nov. 2026.

[7] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms, 2017. arXiv:1708.07747v2.

[8] OpenAI. Codex. AI coding assistant, used 11 Nov. 2026, 2026.

Supplementary Pages: Core Code and Verification

The following excerpts are taken directly from the runnable source. They explain the optimizer partition, measurement boundary, and test isolation. Third-party optimizer internals are called unchanged through PyTorch. Full scripts and raw results accompany the repository. Each training seed resets model initialization and an independent CPU permutation generator; tests check exact repeatability within this environment, not across hardware or library releases.

S1. Parameter routing and optimizer construction

Biases receive zero decay. The input and classifier matrices belong to auxiliary AdamW. The two hidden matrices belong exclusively to Muon; explicit identity checks reject overlaps or omissions.

```
"""Use unmodified PyTorch 2.11 optimizers. Muon is a hybrid treatment."""
import torch


def parameter_routes(model):
    hidden = [(n, p) for n, p in model.named_parameters()
              if n.startswith('body.') and p.ndim == 2]
    hidden_ids = {id(p) for _, p in hidden}
    auxiliary = [(n, p) for n, p in model.named_parameters() if id(p) not in hidden_ids]
    all_ids = [id(p) for _, p in hidden + auxiliary]
    assert len(all_ids) == len(set(all_ids)) == len(list(model.parameters()))
    return hidden, auxiliary


def decay_groups(named, decay):
    return [dict(params=[p for _, p in named if p.ndim >= 2], weight_decay=decay),
            dict(params=[p for _, p in named if p.ndim < 2], weight_decay=0.)]


class OptimizerBundle:
    def __init__(self, optimizers):
        self.optimizers = optimizers

    def zero_grad(self):
        for optimizer in self.optimizers:
            optimizer.zero_grad(set_to_none=True)

    def step(self):
        for optimizer in self.optimizers:
            optimizer.step()


def make_optimizer(model, name, lr, weight_decay=1e-4, aux_lr=1e-3):
    if name == 'muon':
        hidden, auxiliary = parameter_routes(model)
        muon = torch.optim.Muon([p for _, p in hidden], lr=lr,
            weight_decay=weight_decay, momentum=.95, nesterov=True,
            ns_steps=5, adjust_lr_fn='original')
        adamw = torch.optim.AdamW(decay_groups(auxiliary, weight_decay), lr=aux_lr,
                                  betas=(.9, .999), eps=1e-8, foreach=False)
        return OptimizerBundle([muon, adamw])
    groups = decay_groups(list(model.named_parameters()), weight_decay)
    if name == 'sgd':
        opt = torch.optim.SGD(groups, lr=lr, momentum=.9, foreach=False)
    elif name == 'adam':
        opt = torch.optim.Adam(groups, lr=lr, betas=(.9, .999), eps=1e-8,
                               foreach=False, decoupled_weight_decay=False)
    elif name == 'adamw':
        opt = torch.optim.AdamW(groups, lr=lr, betas=(.9, .999), eps=1e-8, foreach=False)
    else:
        raise ValueError(name)
    return OptimizerBundle([opt])
```

S2. Paired initialization, warmup, and measured training

Warmup updates are discarded. After reseeding, a fresh model and optimizer start the actual run. Training and validation are synchronized separately. Additional train-set evaluation and checkpoint writes are outside the cumulative training-plus-validation clock.

```
    seed_all(seed)
    warm = MLP(cfg['hidden']).to(device)
    warm_opt = make_optimizer(warm, name, lr, cfg['weight_decay'], cfg['aux_lr'])
    sync(device)
    t = time.perf_counter()
    for _ in range(cfg['warmup_steps']):
        warm_opt.zero_grad()
        F.cross_entropy(warm(x[:cfg['batch_size']]), y[:cfg['batch_size']]).backward()
        warm_opt.step()
    sync(device)
    warmup_s = time.perf_counter() - t
    del warm, warm_opt
    seed_all(seed)
    model = MLP(cfg['hidden']).to(device)
    initial_hash = model_hash(model)
    opt = make_optimizer(model, name, lr, cfg['weight_decay'], cfg['aux_lr'])
    generator = torch.Generator(device='cpu').manual_seed(seed + 10000)
    checkpoint_dir = ROOT / 'checkpoints' / run_id
    checkpoint_dir.mkdir(parents=True, exist_ok=True)
    best_key = (-1., float('-inf'))
    best_epoch = 0
    cumulative_train = cumulative_wall = 0.
    budget_state = None
    budget_key = (-1., float('-inf'))
    budget_epoch = 0
    if str(device).startswith('cuda'):
        torch.cuda.reset_peak_memory_stats(device)
    rows = []
    for epoch in range(1, epochs + 1):
        # CPU permutation is paired across all treatments, independently of optimizer RNG.
        indices = torch.randperm(len(y), generator=generator).to(device)
        model.train()
        sync(device)
        t = time.perf_counter()
        total_loss = torch.zeros((), device=device)
        for ix in indices.split(cfg['batch_size']):
            opt.zero_grad()
            loss = F.cross_entropy(model(x[ix]), y[ix])
            loss.backward()
            opt.step()
            total_loss += loss.detach() * len(ix)
        sync(device)
        train_s = time.perf_counter() - t
        online_loss = float(total_loss / len(y))
        t = time.perf_counter()
        val_loss, val_acc = evaluate(model, vx, vy)
        sync(device)
        val_s = time.perf_counter() - t
        cumulative_train += train_s
        cumulative_wall += train_s + val_s
        # Extra reporting evaluations/checkpoint I/O are outside the train+validation budget.
        train_loss, train_acc = evaluate(model, x, y)
        key = (val_acc, -val_loss)
        if key > best_key:
            best_key, best_epoch = key, epoch
            torch.save(model.state_dict(), checkpoint_dir / 'best.pt')
        if cumulative_wall <= cfg['time_budget_s'] and key > budget_key:
            budget_key, budget_epoch = key, epoch
            budget_state = copy.deepcopy(model.state_dict())
```

S3. Evaluation and test isolation

Evaluation uses inference mode and divides summed loss and correct predictions by the full sample count. No gradients are computed and no test metric is used in model selection. The final evaluator checks a frozen selection manifest, then reads train, validation, and official test splits for predeclared checkpoints.

```
"""Evaluate a fixed checkpoint; no stochastic layers or optimizer updates."""
import torch
import torch.nn.functional as F

@torch.inference_mode()
def evaluate(model, x, y, batch_size=2048):
    model.eval()
    loss, correct = torch.zeros((), device=x.device), torch.zeros((), device=x.device)
    for start in range(0, len(y), batch_size):
        target = y[start:start+batch_size]
        logits = model(x[start:start+batch_size])
        loss += F.cross_entropy(logits, target, reduction='sum')
        correct += (logits.argmax(1) == target).sum()
    return float(loss / len(y)), float(correct / len(y))
```

```
    frozen = json.loads((ROOT/'results/training_frozen.json').read_text())
    assert frozen['selected_config_sha256'] == sha256(ROOT/'results/selected_config.json')
    cfg = config()
    assert len(frozen['runs']) == 4*len(cfg['seeds'])
    device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
    seed_all(0)
    x,y,vx,vy = load_development(device,cfg['split_seed'])
    tx,ty = load_test(device)
    results = []
    for run_id in frozen['runs']:
        meta = json.loads((ROOT/'results/runs'/run_id/'completed.json').read_text())
        for selection in ['best','final','budget']:
            path = ROOT/'checkpoints'/run_id/f'{selection}.pt'
            if not path.exists():
                if selection != 'budget' or meta['budget_available']:
                    raise FileNotFoundError(f'Reproduce training first: {path}')
                continue
            model = MLP(cfg['hidden']).to(device)
            model.load_state_dict(torch.load(path, map_location=device, weights_only=True))
            trl,tra = evaluate(model,x,y)
            vl,va = evaluate(model,vx,vy)
            tl,ta = evaluate(model,tx,ty)
            results.append(dict(run_id=run_id,optimizer=meta['optimizer'],seed=meta['seed'],
                lr=meta['lr'],selection=selection,epoch=(meta['best_epoch'] if selection=='best
    '
                    else meta['epochs'] if selection=='final' else meta['budget_epoch']),
                train_loss=trl,train_accuracy=tra,val_loss=vl,val_accuracy=va,
                test_loss=tl,test_accuracy=ta,generalization_gap=tra-ta,
                checkpoint_sha256=sha256(path)))
```

S4. Acceptance evidence

Thirteen optimizer/data/schema/resume unit tests passed. The experiment audit verified 12 tuning runs, 12 formal runs, and 308 epoch rows including eight smoke rows. It checks paired initial-weight hashes, equal tuning budgets, checkpoint hashes, validation-based selections, and mean/sample-SD recomputation from raw data. Core files are available under `src/`, `scripts/`, and `tests/`. Run `python scripts/validate_results.py` after evaluation and plotting.